DAY-TO-DAY WORK INTEGRATION

From Yoda Layer to Daily Work

The constitution is the Yoda layer (org-wide). This is the daily-work layer: immediate customer value that also feeds macro behavior.

THE PRINCIPLE: DAILY WORK FEEDS THE CONSTITUTION

The constitution layers (V3-V8) look at the organization as a whole — the Yoda layer. But customers need day-to-day work features that give immediate value AND feed the macro behavior. Every daily-work feature must satisfy two conditions: (1) the customer gets immediate utility (today, not after 90 days), and (2) the feature generates signals that deepen the constitutional layers (capability, intelligence, understanding, model).

The design law (from the immutable constitution): "Every increase in internal intelligence must reduce external complexity." Daily-work features must make the customer's work SIMPLER while making Maestro SMARTER. If a feature adds complexity without feeding the model, it does not belong.

The honest gap, stated plainly: everything above the trust-layer line is mostly UI and orchestration work on primitives Maestro already has. Everything at the trust-layer line is exactly what the audit rounds have been circling. The list below is not "how far away are we" — it is "which half is already true and which half still needs the boring work."

The 10 Day-to-Day Features

#	Feature	What It Does	Effort	Priority	Feeds Constitution
1	Organizational Timeline	Chronological view of everything that happened across all tools	2 days	HIGH	Evidence graph (temporal context)
2	Task & Action-Item Intelligence	Auto-extract to-dos from signals, categorize, prioritize, link to evidence	3 days	HIGH	Learning objects (task LOs)
3	Proactive Daily Briefing (upgraded)	Morning briefing with drafted items: nudge contacts, one-pager for meeting, CEO briefing	2 days	HIGH	Lead engaged + evidence graph
4	Write-Back to Tools (bidirectional)	Create Jira tickets, update CRM, draft email replies, post Slack messages — gated by approval	5 days	HIGH	Signal ingestion (action signals)
5	Writing Assistant (Canvas)	Co-edit AI-drafted documents with live evidence citations. Edits captured as feedback signal	4 days	MEDIUM	Feedback loops + evidence graph
6	Role-Specific Playbooks	Sales (Apollo match + email draft), Marketing (unified ad spend), Product (transcript -> PRD + tickets)	4 days	HIGH	Decision engine + evidence graph
7	Per-Teammate Scoping	Each teammate gets their own daily focus + agenda view. Shared Company DNA keeps consistency	3 days	MEDIUM	Multi-user + tenant isolation
8	Developer IDE Integration (MCP)	Expose evidence graph as MCP server. Search company data from Claude Code / Cursor	2 days	LOW	API surface + adoption
9	Enterprise Trust Layer (final)	SAML fail-closed fix, real tenant isolation, permission-aware indexing, SOC2 checklist	3 days	HIGH	Security (rounds 1-6)
10	Public API Documentation	Stable public contract, Swagger docs, SDK examples	2 days	LOW	Extensibility
TOTAL	**10 features**		**~30 days**		**Every feature feeds the constitution**

Build order: HIGH priority first (#1 Timeline -> #9 Trust Layer -> #3 Briefing -> #2 Tasks -> #4 Write-Back -> #6 Playbooks), then MEDIUM (#5 Canvas -> #7 Per-Teammate), then LOW (#8 MCP -> #10 API Docs). Total ~30 days. But build #1, #9, #3 first — they give immediate value AND unblock the pilot.

#1 Organizational Timeline — "what happened" without hunting

What Nerve did

Nerve tracked a running record of everything that happened across connected tools in order, so a user could reconstruct "what happened" without hunting across 5 dashboards.

Current codebase gap

The evidence graph already has event timestamps. A drill-down timeline tab exists for individual entities. But there is no ORG-WIDE chronological timeline endpoint — no GET /api/oem/timeline that returns a paginated, filterable chronological view of ALL signals across ALL providers. The gap: the data exists; the API does not.

Build

CREATE GET /api/oem/timeline endpoint. Paginated (?page=1&limit;=50), filterable by provider, domain, actor, signal_type, date range. Returns signals in chronological order with: timestamp, provider, actor, artifact, signal_type, domain, description (humanized). CREATE static/js/timeline.js — a new surface (command-palette only, NOT in sidebar). Renders a vertical timeline of events, grouped by day, with provider icons and domain tags. Filterable by provider/domain/actor. MODIFY static/js/maestro.js — add "Timeline" to command palette.

API contract

GET /api/oem/timeline?page=1&limit;=50&provider;=github&domain;=payments returns: { "events": [{"timestamp": "...", "provider": "github", "actor": "priya.m@acme.com", "artifact": "github:acme/payments-edge/pull/447", "signal_type": "pr_opened", "domain": "payments", "description": "Priya opened a pull request in payments-edge"}], "total": 47, "page": 1, "limit": 50 }. Must support filtering by provider, domain, actor, signal_type, and date range (since/until).

Acceptance test

1) GET /api/oem/timeline returns events array (10+ for demo seed). 2) Each event has timestamp, provider, actor, artifact, signal_type, domain, description. 3) Filtering works (?provider=github returns only GitHub events). 4) Pagination works (?page=2 returns different events). 5) Timeline surface accessible via Ctrl+K (NOT in sidebar). 6) V5 litmus: no new sidebar item. 7) Feeds constitution: evidence graph gains temporal context ("this contradicts something raised in March").

Effort

2 days (1 day API + 0.5 day frontend + 0.5 day tests)

Priority

HIGH — build first. Immediate value + unblocks temporal reasoning.

#2 Task & Action-Item Intelligence — auto-extract to-dos from signals

What Nerve did

Nerve proactively tracked to-dos and updates across every team and project with automatic categorization and prioritization, not a flat list.

Current codebase gap

gps.py has some task-like logic. But there is no TASK EXTRACTION step in the ingestion pipeline — no mechanism to scan incoming signals (Slack messages, Jira issues, meeting transcripts) for action items ("priya to review by Friday", "carlos will draft the RFC"). The gap: signals are ingested as events, not as tasks. The decision engine produces RECOMMENDATIONS, not TASKS.

Build

CREATE backend/maestro_oem/task_extraction.py — the TaskExtractor. Scans signal metadata (text, title, description) for action-item patterns: "X to Y by Z", "X will Y", "action: Y", "todo: Y", "follow up on Y". Each extracted task has: description, assignee (from actor/entity), due_date (if mentioned), source_signal_id, priority (inferred from signal urgency), status (open). Tasks are stored as learning objects with type="task". CREATE GET /api/oem/tasks endpoint (returns open tasks, filterable by assignee/domain/priority). MODIFY backend/maestro_api/oem_state.py live_ingest() — call TaskExtractor after each signal batch. MODIFY static/js/today.js — add a "Your tasks" section showing open tasks with assignee, due date, and source-signal link.

API contract

GET /api/oem/tasks?assignee=priya.m@acme.com&status;=open returns: { "tasks": [{"id": "task-xxx", "description": "Review circuit breaker PR", "assignee": "priya.m@acme.com", "due_date": null, "priority": "high", "source_signal": "slack:C-123/p-1", "status": "open", "created_at": "..."}], "total": 3 }. Tasks must reference the source signal (evidence chain).

Acceptance test

1) GET /api/oem/tasks returns tasks extracted from real signals (not hardcoded). 2) Each task has description, assignee, source_signal, priority, status. 3) Tasks are created during ingestion (not on API call). 4) TODAY shows "Your tasks" section. 5) V5 litmus: no new panel — enhances TODAY. 6) Feeds constitution: learning objects with type="task" enrich the model.

Effort

3 days (1.5 days extraction engine + 0.5 day API + 0.5 day frontend + 0.5 day tests)

Priority

HIGH — immediate value (customers see their to-dos surfaced automatically).

#3 Proactive Daily Briefing (upgraded) — drafted items, not just summaries

What Nerve did

Every morning, Nerve delivered a briefing like "scanned 214 emails and 11 meetings overnight — three things need you", each item already DRAFTED: a nudge to a contact who had gone quiet, a one-pager built from past threads for an unfamiliar meeting, an inbound lead pre-tagged by domain.

Current codebase gap

today.js already renders a morning brief (one decision, one opportunity, one risk, one learning, one prediction). But the items are SUMMARIES, not DRAFTS. The gap: the brief says "Address the bottleneck" but does not DRAFT the nudge email, the one-pager, or the lead tag. The executive function engine (V5 #2) produces a plan, but the morning brief does not call it. The gap: the brief should include DRAFTED artifacts (email draft, meeting prep doc, lead summary) linked to evidence.

Build

UPGRADE backend/maestro_oem/sowhat.py + executive_function.py — add a `draft_morning_items()` method that, for each brief item, generates a DRAFTED artifact: (1) for decisions: a drafted email to the decision owner ("Hi Jane, the payments bottleneck needs your attention..."), (2) for opportunities: a one-pager built from evidence graph nodes ("Payments bottleneck: 3 items gated, 5-day avg delay, evidence: PR #447, Jira EMEA-1247..."), (3) for risks: a risk summary with evidence chain. MODIFY GET /api/oem/ceo-briefing — each item gets a `draft` field with the drafted text. MODIFY static/js/today.js — each brief item shows a "View draft" link that opens the drafted artifact in the drill-down modal.

API contract

GET /api/oem/ceo-briefing returns items with a `draft` field: { "title": "...", "draft": "Hi Jane, The payments-edge bottleneck has gated 3 items for 5 days... Evidence: PR #447 (opened Nov 12), Jira EMEA-1247 (P1)... Recommended action: Redistribute approval authority... — Maestro" }. Draft must reference real evidence (signal IDs, PR numbers, Jira tickets).

Acceptance test

1) GET /api/oem/ceo-briefing items have a `draft` field (non-empty for at least 1 item). 2) Draft references real evidence (signal IDs, artifact references). 3) TODAY shows "View draft" link on brief items. 4) Draft opens in drill-down modal. 5) V5 litmus: no new panel — enhances TODAY. 6) Feeds constitution: drafts are evidence-backed, strengthening the evidence graph.

Effort

2 days (1 day draft-generation engine + 0.5 day API + 0.5 day frontend)

Priority

HIGH — immediate value (executive gets drafted emails/docs, not just summaries).

#4 Write-Back to Tools (bidirectional) — the biggest actual build item

What Nerve did

Nerve wrote docs, updated CRM records, created actual Jira tickets, and drafted email replies — all through direct write access to those apps, with the user approving or editing before anything sent.

Current codebase gap

ALL of Maestro's importers are read-only. github_importer.py fetches PRs/issues but cannot create them. jira_importer.py fetches issues but cannot create/update them. slack provider reads messages but cannot post. The gap: Maestro can RECOMMEND actions but cannot EXECUTE them. The executive function engine produces a plan; the user must manually execute each step. This is the actual gap between "ingests signals" and "does your actual work" — it is a real build item, not a small one.

Build

CREATE backend/maestro_oem/writeback/ directory with one writeback module per provider: (1) jira_writeback.py — create issues, update status, add comments (via Jira REST API POST), (2) github_writeback.py — create PR review comments, create issues (via GitHub REST API POST), (3) slack_writeback.py — post messages to channels (via Slack Web API chat.postMessage), (4) gmail_writeback.py — draft emails (via Gmail API users.drafts.create — does NOT send, just drafts). Each writeback module has: execute(action, params) -> result, and requires_approval=True (user must approve before write-back). CREATE POST /api/oem/writeback endpoint (accepts provider, action, params; returns preview; user approves via second POST). MODIFY static/js/today.js — the "Prepare" button on the executive function plan gets a second button: "Execute" (calls writeback, shows preview, user approves).

API contract

POST /api/oem/writeback with {"provider": "jira", "action": "create_issue", "params": {"project": "EMEA", "summary": "Address payments bottleneck", "description": "..."}} returns: {"preview": {"url": "https://...", "summary": "..."}, "requires_approval": true, "writeback_id": "wb-xxx"}. Second POST /api/oem/writeback/wb-xxx/approve executes the write-back and returns the result. Gmail writeback ONLY drafts (does not send — user sends manually from Gmail).

Acceptance test

1) POST /api/oem/writeback returns a preview (not executed). 2) POST /api/oem/writeback/{id}/approve executes and returns result. 3) Jira: creates a real issue (verified via mock or test Jira). 4) Gmail: creates a DRAFT (not sent — user sends manually). 5) All write-backs require approval (no autonomous execution without governance mode = autonomous). 6) V5 litmus: "Execute" button enhances TODAY. 7) Feeds constitution: write-back actions generate new signals (action signals) that feed the model.

Effort

5 days (1 day per provider writeback module x 4 + 0.5 day API + 0.5 day frontend + 0.5 day tests)

Priority

HIGH — this is THE gap between "advises" and "does work." Biggest build item but highest impact.

#5 Writing Assistant (Canvas) — co-edit AI-drafted documents with live evidence

What Nerve did

Nerve's "Canvas"/"Live Draft" mode let a user co-edit AI-drafted documents in real time. Users specifically called this the best writing experience they had used, better than raw ChatGPT because it already knew company context.

Current codebase gap

No document editor exists in Maestro. The executive function engine produces DRAFTS (briefing memos, email drafts) but they are read-only text in the drill-down modal. The gap: the user cannot EDIT the draft, and edits are not captured as feedback. The loop: Maestro drafts -> user edits -> edit captured as feedback signal -> model learns from the edit.

Build

CREATE static/js/canvas.js — a rich-text editing panel that loads a drafted artifact (from executive_function.py or sowhat.py), allows the user to edit it inline, and shows cited evidence inline (as footnotes/links). When the user saves, the diff between Maestro's draft and the user's edit is captured as a feedback signal (type: "draft_feedback") and fed into the learning loop. CREATE GET /api/oem/canvas/{draft_id} endpoint (returns draft text + evidence citations). CREATE POST /api/oem/canvas/{draft_id}/save endpoint (accepts edited text, computes diff, creates feedback signal). MODIFY static/js/today.js

— "View draft" link opens the Canvas editor instead of the drill-down modal.

API contract

GET /api/oem/canvas/{draft_id} returns: { "draft_text": "...", "evidence_citations": [{"index": 1, "signal_id": "...", "text": "PR #447"}], "created_at": "..." }. POST /api/oem/canvas/{draft_id}/save accepts: { "edited_text": "..." }. Returns: { "ok": true, "feedback_signal_id": "sig-xxx", "diff_summary": "User changed 3 paragraphs, removed 1 evidence citation, added 1 new section." }.

Acceptance test

1) Canvas editor loads a draft with evidence citations visible. 2) User can edit the text. 3) On save, a feedback signal is created (verified: signal appears in model). 4) The diff between draft and edit is captured. 5) V5 litmus: Canvas is command-palette accessible (not in sidebar). 6) Feeds constitution: edit feedback closes the learning loop (Maestro learns how the org writes).

Effort

4 days (2 days canvas editor + 1 day API + 0.5 day feedback-signal integration + 0.5 day tests)

Priority

MEDIUM — high value but not blocking the pilot. Build after #1-#4.

#6 Role-Specific Playbooks — sales, marketing, product templates

What Nerve did

Nerve had role-specific playbooks: sales (Apollo match + email draft with SOC2 talking point from call transcript), marketing (unified ad spend view across Google/Meta/LinkedIn/GA4/Stripe), product (call transcript -> PRD + tickets with old concerns flagged).

Current codebase gap

No playbooks exist. The decision engine produces general recommendations. The gap: no ROLE-SPECIFIC templates that format the same evidence differently for a sales rep vs a product manager vs a marketing lead. These are not new engines — they are thin layers over decision.py that format output for specific roles.

Build

CREATE backend/maestro_oem/playbooks/ directory with 3 playbook modules: (1) sales_playbook.py — matches CRM contacts against evidence-graph patterns ("this contact went quiet after the pricing call"), drafts outreach emails with talking points from call transcripts, (2) marketing_playbook.py — unifies ad-spend signals across providers into a single ROI view ("Google ad set X burns budget at 3x the cost-per-lead of LinkedIn"), (3) product_playbook.py — takes a call transcript and returns a PRD outline + Jira tickets with old unresolved concerns flagged. Each playbook is a thin layer: query evidence graph + format output for the role. CREATE GET /api/oem/playbook/{role}?context=... endpoint. CREATE static/js/playbook.js — a surface (command-palette only) that lets the user select a role and context.

API contract

GET /api/oem/playbook/sales?context=globex-renewal returns: { "items": [{"type": "outreach_email", "contact": "raj@globex.com", "draft": "Hi Raj, Following up on our Q4 renewal discussion... Talking point: SOC2 compliance (raised in Nov 12 call)...", "evidence": ["gmail:msg-001", "cal:event-001"]}], "summary": "1 contact needs outreach. 1 talking point from call transcript." }. GET /api/oem/playbook/product?context=auth-refactor returns: { "prd_outline": "...", "tickets": [{"summary": "...", "description": "..."}], "unresolved_concerns": ["Legal raised security concern in Oct (unresolved)"] }.

Acceptance test

1) GET /api/oem/playbook/sales returns drafted outreach with evidence. 2) GET /api/oem/playbook/product returns PRD outline + tickets + unresolved concerns. 3) GET /api/oem/playbook/marketing returns unified ROI view. 4) Playbook surface accessible via Ctrl+K (NOT in sidebar). 5) V5 litmus: no new sidebar item. 6) Feeds constitution: playbooks generate new signal types (outreach, PRD, ticket) that feed the model.

Effort

4 days (1 day per playbook + 0.5 day API + 0.5 day frontend)

Priority

HIGH — role-specific value drives adoption. Sales rep sees drafted emails = immediate ROI.

#7 Per-Teammate Scoping — each teammate gets their own focus

What Nerve did

Nerve ran for a single operator first; once a company grew, every teammate got their own daily focus and their own dedicated agent team, while a shared "Company DNA" kept every agent consistent.

Current codebase gap

multiuser.py exists (multi-user support). Tenant isolation middleware exists (V6 wiring fix). But there is no PER-USER VIEW of the morning brief, tasks, or recommendations. The brief is org-wide. The gap: a sales rep should see their tasks and leads; an engineer should see their PRs and bottlenecks; the CEO sees everything. The brief is not scoped per user.

Build

UPGRADE backend/maestro_api/oem_state.py — add a `user_scope` parameter to brief/task/recommendation generation. When a user is logged in (via auth middleware), filter signals/tasks/recommendations to those involving the user (as actor, assignee, or team member). The org-wide view (CEO) remains available. CREATE GET /api/oem/ceo-briefing?user=priya.m@acme.com endpoint (returns user-scoped brief). MODIFY static/js/today.js — if a user is logged in, show "Your day" (user-scoped) instead of "Good morning" (org-wide). Add a toggle: "Your day" / "Org view".

API contract

GET /api/oem/ceo-briefing?user=priya.m@acme.com returns items scoped to priya: tasks where priya is assignee, recommendations about priya's domains, risks in priya's teams. GET /api/oem/ceo-briefing (no user param) returns the org-wide brief (CEO view).

Acceptance test

1) GET /api/oem/ceo-briefing?user=X returns items involving X (not org-wide). 2) GET /api/oem/ceo-briefing (no user) returns org-wide. 3) TODAY shows "Your day" when user is logged in, "Org view" toggle available. 4) V5 litmus: toggle enhances TODAY (no new panel). 5) Feeds constitution: per-user signals enrich the multi-user model.

Effort

3 days (1.5 days user-scoping logic + 0.5 day API + 0.5 day frontend + 0.5 day tests)

Priority

MEDIUM — important for scaling beyond a single user, but pilot can start with org-wide view.

#8 Developer IDE Integration (MCP) — search company data from Claude Code/Cursor

What Nerve did

Nerve connected directly to Claude Code and Cursor, so a developer could search and act on company data without leaving their IDE.

Current codebase gap

No MCP server exists. Maestro's API is FastAPI (REST). The gap: developers cannot query the evidence graph from their IDE. They must open a browser and navigate to Maestro. An MCP (Model Context Protocol) server would expose the evidence graph as a tool that Claude Code/Cursor can call directly.

Build

CREATE backend/mcp_server.py — a lightweight MCP server that exposes 3 tools: (1) search_org(query) — searches the evidence graph (calls /api/oem/ask internally), (2) get_timeline(domain, since) — returns timeline events for a domain, (3) get_explanation(question) — returns an explanation chain. The MCP server runs as a separate process (python -m maestro_mcp.server) and communicates via stdio (MCP protocol). Package as pip-installable (maestro-mcp).

API contract

MCP server exposes 3 tools: search_org, get_timeline, get_explanation. Each returns JSON. The server runs via stdio (standard MCP transport). A developer using Claude Code can call search_org("payments bottleneck") and get evidence-graph results without leaving the IDE.

Acceptance test

1) MCP server starts and responds to tool calls. 2) search_org returns real evidence-graph data. 3) get_timeline returns chronological events. 4) get_explanation returns a causal chain. 5) V5 litmus: no UI change (backend-only). 6) Feeds constitution: developer queries generate signals (adoption signals) that feed the model.

Effort

2 days (1 day MCP server + 0.5 day packaging + 0.5 day tests)

Priority

LOW — high leverage for this specific repo (built with Claude Code), but not blocking the pilot.

#9 Enterprise Trust Layer (final) — SAML, tenant isolation, permission-aware indexing

What Nerve did

Nerve had SOC 2 compliance, SAML/SSO, permission-aware indexing across 100+ connected apps, and an explicit no-training-on-customer-data policy.

Current codebase gap

SAML exists (saml.py, 236 lines) but has the fail-open issue from round 4 (signature present but python3-saml missing = accept with warning). Tenant isolation middleware exists but is a no-op in single-tenant mode. The gap: SAML must fail CLOSED (reject unsigned/ unverifiable responses). Tenant isolation must enforce per-user data scoping. Permission-aware indexing (only ingest signals the user has permission to see) does not exist.

Build

FIX backend/maestro_auth/saml.py — when python3-saml is not installed, RAISE SAMLError (do not accept). This is the same fix pattern as the OIDC algorithm-injection fix from round 5. FIX backend/maestro_auth/security.py TenantIsolationMiddleware — enforce per-user scoping even in single-tenant mode (filter signals by user's team/domain membership). CREATE backend/maestro_oem/permission_indexer.py — before ingesting a signal, check if the signal's source (channel, repo, project) is accessible to the user who connected it. CREATE docs/SOC2_CHECKLIST.md — the compliance checklist (data encryption, audit trail, access controls, data retention, incident response).

API contract

1) SAML: GET /api/auth/saml/{provider}/callback with an unsigned response returns 401 (not 200). 2) Tenant isolation: user A cannot see user B's signals (verified via API). 3) Permission-aware: signals from private channels are not ingested unless the user has access. 4) SOC2 checklist exists in docs/.

Acceptance test

1) SAML fail-closed: verified by submitting unsigned SAML response -> 401. 2) Tenant isolation: user A GET /api/oem/tasks?user=A does not return user B tasks. 3) Permission-aware: private channel signals are filtered. 4) SOC2 checklist is comprehensive (10+ items). 5) Feeds constitution: trust layer is the prerequisite for enterprise pilot.

Effort

3 days (0.5 day SAML fix + 1 day tenant isolation enforcement + 1 day permission-aware indexer + 0.5 day SOC2 docs)

Priority

HIGH — this is the literal checklist for enterprise-ready. Build before pilot.

#10 Public API Documentation — stable contract, Swagger, SDK examples

What Nerve did

Nerve had a full REST API that let other tools build on top of it rather than being locked into its own UI.

Current codebase gap

Maestro's FastAPI backend already has auto-generated OpenAPI docs at /docs. But the API contract is not stable (endpoints change between versions), there is no SDK, and the docs are auto-generated (not curated). The gap: a customer building on top of Maestro needs a STABLE, DOCUMENTED, VERSIONED API contract.

Build

CREATE docs/API_CONTRACT.md — curated API documentation with stable endpoints, request/response examples, error codes, rate limits, and versioning policy. ADD API versioning: prefix all routes with /api/v1/ (currently /api/). CREATE examples/ directory with Python and JavaScript SDK examples (5-10 lines each for common operations: get brief, ask question, get timeline, create task). ADD /api/v1/health endpoint that returns API version, build hash, and status.

API contract

GET /api/v1/health returns: { "version": "1.0.0", "build": "1959bdf", "status": "ok", "uptime_seconds": 3600 }. docs/API_CONTRACT.md exists with 10+ endpoint examples. examples/ directory has Python and JS SDK examples.

Acceptance test

1) /api/v1/health returns version + status. 2) docs/API_CONTRACT.md has 10+ documented endpoints. 3) examples/ has Python + JS code samples. 4) All routes prefixed with /api/v1/ (backward compat: /api/ redirects to /api/v1/). 5) V5 litmus: no UI change. 6) Feeds constitution: stable API enables ecosystem (other tools build on Maestro, generating adoption signals).

Effort

2 days (1 day API versioning + 0.5 day docs + 0.5 day examples)

Priority

LOW — important for ecosystem, but not blocking the pilot.

Build Order

Step	Feature	Effort	Priority	Why This Order
1	#9 Enterprise Trust Layer	3 days	HIGH	SAML fail-closed + tenant isolation = prerequisite for pilot. Build FIRST.
2	#1 Organizational Timeline	2 days	HIGH	Immediate value (customer sees "what happened"). Data already exists. Unblock
3	#3 Proactive Daily Briefing (upgraded)	2 days	HIGH	Immediate value (drafted emails/docs, not just summaries). CEO opens Maestro
4	#2 Task & Action-Item Intelligence	3 days	HIGH	Immediate value (to-dos surfaced automatically). Feeds learning objects.
5	#4 Write-Back to Tools	5 days	HIGH	THE gap between "advises" and "does work." Biggest build item but highest imp
6	#6 Role-Specific Playbooks	4 days	HIGH	Role-specific value drives adoption (sales rep sees drafted emails = ROI).
7	#7 Per-Teammate Scoping	3 days	MEDIUM	Scaling beyond single user. Build after pilot proves single-user value.
8	#5 Writing Assistant (Canvas)	4 days	MEDIUM	High value but not blocking pilot. Build after core daily-work features.
9	#8 Developer IDE Integration (MCP)	2 days	LOW	High leverage for this repo but not blocking pilot.
10	#10 Public API Documentation	2 days	LOW	Important for ecosystem but not blocking pilot.
TOTAL	**10 features**	**~30 days**		**Build #9 + #1 + #3 first (7 days). Then pilot-ready with immediate valu**

Strict Rules

Rule	Enforcement
1. Build #9 (Trust Layer) FIRST	SAML fail-closed + tenant isolation = prerequisite for pilot. No pilot without it.
2. Every feature feeds the constitution	Each feature must generate signals that deepen the model. If a feature does not feed the evidence graph, it does not belor
3. V5 litmus test retained	No new sidebar items (stays at 4). No organ names. No new panels. Every feature enhances an existing surface or is com
4. Write-back requires approval (governance)	No autonomous write-back without governance mode = autonomous (V7 Spec #2). User must approve every write-back
5. Gmail writeback ONLY drafts	Never sends email automatically. Creates a draft in Gmail. User sends manually.
6. Permission-aware indexing	Only ingest signals from channels/repos/projects the user has access to. No private data ingestion without permission.
7. 5-point checklist (all YES)	1) Full acceptance test. 2) Application not existence. 3) Full test suite. 4) Live API. 5) UI simpler + feeds constitution.
8. No silent skips	If a feature is deferred, say so explicitly. Do NOT claim "all 10 delivered" if only 7 are built.
9. Run FULL test suite	423+ tests. Report exact count. No subsets.
10. Constitution is frozen	These features do NOT change the constitution. They are daily-work features that FEED the constitutional layers. The c

Final Charge

DAILY WORK FEEDS THE CONSTITUTION. BUILD #9 FIRST. THEN #1 + #3. THEN PILOT.

The constitution is the Yoda layer — org-wide, immutable, frozen. These 10 features are the daily-work layer — immediate customer value that feeds the constitution. Build #9 (Trust Layer) first — SAML fail-closed + tenant isolation is the prerequisite for the pilot. Then #1 (Timeline) and #3 (Upgraded Briefing) — 7 days total for the first 3 features. That makes the product pilot-ready with immediate value.

Then #2 (Tasks), #4 (Write-Back), #6 (Playbooks) — the core daily-work features. Then #7 (Per-Teammate), #5 (Canvas), #8 (MCP), #10 (API Docs) — the scaling features.

The honest gap: everything above the trust-layer line is mostly UI and orchestration on primitives Maestro already has. Everything at the trust-layer line is exactly what the audit rounds have been circling. Build the trust layer. Then the daily-work features. Then ship the pilot. The org becomes more capable, not more dependent. That is the moat. That is the product. Build it.